

AI-Ready Context

watsonx 위에 올린 설명가능한 Agentic RAG + Knowledge Graph

질문마다 벡터 · 그래프 · SQL 도구로 자동 라우팅하고
답의 근거를 그대로 보여주는 데모

Kiyeon Jeon · AI Engineer, IBM Client Engineering Korea
IBM TechXchange Korea 워크숍

발표 노트: "안녕하세요, IBM Client Engineering Korea의 AI Engineer 전기연입니다. 오늘 보여드릴 건 RAG를 한 단계 더 끌어올린 형태입니다. 질문에 따라 적합한 검색 도구를 에이전트가 스스로 고르고, 그 근거를 화면에 펼쳐 보여줍니다. 전부 watsonx 스택과 OpenShift 위에 올렸습니다."

목차

1. 배경 — 왜 Agentic RAG + Knowledge Graph인가
2. 아키텍처 & 데이터 — 스토어 구성 · 동작 방식 · 문서 적재 · 데모 코퍼스
3. 데모 — ① 3경로 라우팅(vector/graph/sql) · ② No-code 즉석 적재 · ③ 설명가능성
4. 심화 — KG를 그래프DB가 아닌 NoSQL(AstraDB)에 담은 법
5. 운영 — 검색·KG 품질 개선 · 배포 · 보안 · 한계
6. 정리 & Q&A

발표 노트: "순서는 이렇습니다. 먼저 왜 만들었는지 배경을 짧게 말씀드리고, 아키텍처와 데이터 구성을 본 뒤, 라이브 데모로 세 경로와 즉석 적재를 보여드립니다. 그다음 자주 받는 질문인 '왜 그래프DB가 아니냐'를 심화로 다루고, 운영 관점(품질·배포·보안·한계)으로 마무리합니다."

왜 이걸 만들었나

- RAG의 한계 — 벡터 검색 하나로는 못 푸는 질문이 많다
 - "총 몇 개 문서가 있나?" → 검색이 아니라 집계(SQL)
 - "A 법과 B 기관의 관계는?" → 의미 유사도가 아니라 관계(그래프)
- 신뢰 문제 — LLM 답이 맞는지 어떻게 믿나? → 근거(grounding)를 보여줘야 한다
- 운영 현실 — 폐쇄망/규제 산업은 SaaS LLM을 그냥 못 쓴다 → watsonx로 거버넌스

→ "질문에 맞는 도구를 고르고, 근거를 보여주는" 에이전틱 RAG

발표 노트: "현장에서 RAG PoC를 하면 꼭 막히는 지점이 있습니다. 집계형 질문, 관계형 질문, 그리고 '이 답 믿어도 되냐'는 질문이죠. 이 세 가지를 정면으로 다룬 게 이 데모입니다."

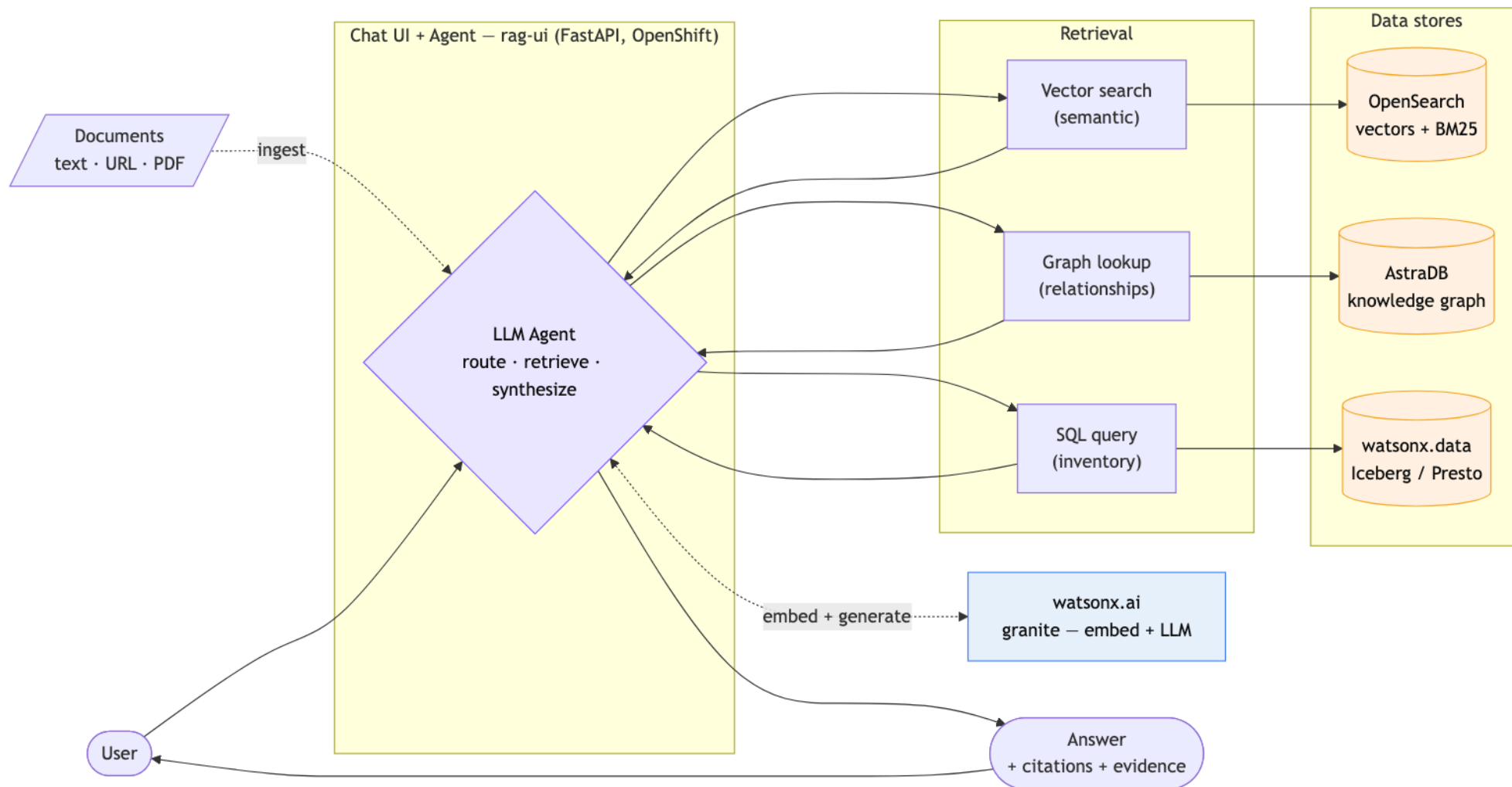
무엇을 보여주나 (3가지 포인트)

1. 에이전트 라우팅 — granite LLM이 질문을 보고 `vector / graph / sql` 도구를 선택
→ 어떤 도구를 썼는지 **tool-trace** 배지로 표시
2. 설명가능성(Explainability) — 답변마다 인용 + 근거 패널
→ 검색된 청크 · KG 서브그래프 · SQL 결과를 펼쳐서 확인
3. No-code 즉석 적재 — UI에서 문서를 텍스트/URL/PDF로 추가
→ 답이 즉시 바뀌고, 삭제하면 원복
→ "grounding 있으면 정확, 없으면 환각"을 클릭만으로 시연

발표 노트: "딱 세 가지만 기억하시면 됩니다. 라우팅, 설명가능성, 그리고 즉석 적재. 마지막 세 번째가 오늘의 하이라이트입니다."

아키텍처 (High-Level)

Agentic RAG + Knowledge Graph – High-Level



데이터 스토어 — 한 질문, 세 가지 검색

스토어	무엇을 담나	어떤 질문에 강한가
OpenSearch	청크 + 임베딩(768d) + 원문 텍스트	"가명정보란 무엇인가" (의미 검색)
AstraDB (KG)	엔티티 · 관계(엣지)	"접근매체와 감독기관의 관계" (관계)
Iceberg / Presto	문서 인벤토리 테이블	"총 몇 개 문서, 각 청크 수" (집계)

- 핵심: 같은 문서를 세 가지 형태로 저장해 두고, 질문에 맞춰 골라 읽는다
- OpenSearch는 벡터(의미) + BM25(정확 매칭)를 **RRF로 융합** → 약어·법령명에 강함

발표 노트: "같은 문서를 세 가지 모양으로 저장해 둡니다. 줄글, 관계망, 표. 질문이 어떤 모양을 원하는지에 따라 에이전트가 적절한 걸 꺼내 씁니다."

동작 방식 — 질문이 들어오면

질문

- [라우터] granite가 도구 선택 {vector, graph, sql}
- vector : OpenSearch 하이브리드 (kNN + BM25, RRF 융합, k=8)
- graph : AstraDB KG 로드 → 벡터시드 + 관련도 정렬 → 1홉 서브그래프
- sql : Presto로 Iceberg corpus 테이블 집계
- [합성] granite가 컨텍스트로 답변 생성 + 인용
- UI : 답변 + tool-trace 배지 + 근거 패널

- 라우팅·검색·합성 전부 **watsonx.ai granite-3-8b** 직접 호출 (Langflow 미사용 — 단순한 흐름엔 직접 구현이 더 명료)
- 답변 언어는 질문 언어를 따름 (한글 질문 → 한글 답변)

발표 노트: "내부 흐름입니다. 라우터가 도구를 고르고, 검색하고, 합성합니다. 전부 granite 한 모델로 처리합니다. 흐름이 단순할 땐 Langflow 같은 비주얼 빌더보다 직접 구현이 더 투명합니다."

문서 적재 — 한 번에 4개 저장소

```
ingest_source(문서)    doc_id = sha1(source)
├─ OpenSearch          : 청크 + granite 임베딩(768d)      ← vector
├─ AstraDB kg          : granite 추출 엔티티/엣지          ← graph
├─ AstraDB doc_registry: 제목·소스·해시·카운트            ← 추적/증분
└─ Iceberg corpus      : 문서 인벤토리 1행                ← sql
```

- 멍등 upsert (doc_id 기준) + **content-hash** 스킵 → 재적재해도 중복 없음
- **CronJob rag-reindex** (30분): 등록된 소스 재적재, 바뀐 것만 재처리
- 문서 파싱은 **docling-serve** (텍스트 / URL / PDF·DOCX → markdown)

발표 노트: "문서 하나를 넣으면 같은 ID로 네 군데에 동시에 들어갑니다. 벡터, 그래프, 추적 레지스트리, SQL 인벤토리. 해시 기반이라 같은 문서를 또 넣어도 중복되지 않습니다."

데모 코퍼스 — 한글 금융·컴플라이언스

직접 작성한 한글 금융 법령 6종 + 영문 README 2종 (혼합)

- 법령 6종 — 개인정보 보호법 · 신용정보법 · 마이데이터(본인신용정보관리업) · 전자금융거래법 · 전자금융감독규정 · 특정금융정보법(자금세탁방지)
- 영문 2종 — Docling README · Langflow README (*URL 적재*)
- 왜 금융 법령? 법령 간 교차참조가 많아 지식그래프가 "의미를 갖는" 도메인
- 한글 + 영문 혼합 → 다국어 임베딩 / 한글 BM25 동시 시연

발표 노트: "코퍼스를 일부러 한국 금융 법령으로 골랐습니다. 법끼리 서로를 인용하는 관계가 많아서, 지식그래프가 진짜 쓸모 있는 도메인이거든요. 영문 문서도 섞어서 다국어 처리도 같이 보여줍니다."

데모 1 — 세 가지 검색 경로

경로	예시 질문	보여줄 것
vector	"가명정보란 무엇이고 동의 없이 쓸 수 있는 목적은?"	인용칩 + 검색 청크(유사도 바)
graph	"접근매체와 감독기관의 관계를 지식그래프로 보여줘"	엔티티 칩 + 엣지 트리플
sql	"인덱싱된 문서는 총 몇 개이고 각각 청크 수는?"	Iceberg 테이블 행
hybrid	"자금세탁 의심거래는 어디 보고하고, VASP 의무는?"	vector+graph 배지 동시 점등

→ 각 답변에서 **tool-trace** 배지와 근거 패널을 펼쳐 보여준다

발표 노트: "첫 데모는 세 경로를 하나씩 보여줍니다. 질문을 바꿀 때마다 위쪽 배지가 어떤 도구를 썼는지 알려주고, 패널을 열면 실제 근거가 나옵니다."

데모 1a — vector 경로

 Lakehouse Assistant
Agentic RAG · Knowledge Graph · watsonx

OpenSearchAstraDBwatsonx.datawatsonx.ai

☰ 코퍼스

가명정보란 무엇이고 동의 없이 쓸 수 있는 목적은?

가명정보란 추가 정보 없이는 특정 개인을 알아볼 수 없도록 가명처리한 정보이다. 가명정보는 통계작성, 과학적 연구, 공익적 기록보존 목적으로 정보주체의 동의 없이 처리할 수 있다. 가명정보 개념은 신용정보법의 가명정보·익명정보 체계와 연계되어 마이데이터·데이터 결합에 활용된다.

개인정보 보호법 #1개인정보 보호법 #0신용정보법 #0마이데이터 (본인신용정보관리업) #0전자금융감독규정 #0신용정보법 #1특정금융정보법 (자금세탁방지) #0특정금융정보법 (자금세탁방지) #1

TOOLS● vector search● knowledge graph● sql / iceberg

● Retrieved passages 8

개인정보 보호법 #1 문서 0.033

없이는 특정 개인을 알아볼 수 없도록 가명처리한 정보이다. 가명정보는 통계작성, 과학적 연구, 공익적 기록보존 목적으로 정보주체의 동의 없이 처리할 수 있다. 가명정보 개념은 **신용정보법**의 가명정보·익명정보 체계와 연계되어 마이데이터·데이터 결합에 활용된다. ## 정보주체의 권리 정보주체는 자신의 개인정보에 대하여 열람, 정정·삭제, 처리정지, 전송요구(개인정보 이동권)를 요구할 권리를 가진다. 전송요구권은 **마이데이터** 서비스의 법적 기반 중 하나이다. > 관련 문서: 신용정보법, 마이데이터(본인신용정보관리업), 전자금융거래법, 특정금융정보법

개인정보 보호법 #0 문서 0.033

개인정보 보호법 (PIPA) 핵심 정리 ## 목적과 적용 대상 개인정보 보호법은 개인정보의 처리 및 보호에 관한 사항을 정하여 개인의 자유와 권리를 보호하는 것을 목적으로 한다. 살아 있는 개인에 관한 정보로서 성명·주민등록번호 등으로 특정 개인을 알아볼 수 있는 정보를 **개인정보**라 한다. 개인정보 파일을 운용하기 위하여 개인정보를 처리하는 자를 **개인정보처리자**라 하고, 처리의 대상이 되는 사람을 **정보주체**라 한다. ## 감독기구 이 법의 소관 감독기구는 **개인정보보호위원회(개인정보위)**이다. 개인정보보호위원회는 정책 수립, 침해 조사, 과징금·과태료 부과를 담당한다. 금융 분야의 개인정보는 **신용정보법**이 우선 적용되며, 그 감독은 **금융위원회**와 **금융감독원**이 수행한다. ## 핵심 의무 - **수집·이용 동의**: 개인정보처리자는 정보주체의 동의를 받아 개인정보를 수집·이용한다. 목적 외 이용은 원칙적으로 금지된다. - **유출 통지·신고**: 개인정보가 유출되었을 때 개인정보처리자는 정보주체에게 지체 없이 통지하고, 일정 규모 이상은 **72시간** 이내에 개인정보보호위

데모 1b — graph 경로

 Lakehouse Assistant
Agentic RAG · Knowledge Graph · watsonx

OpenSearchAstraDBwatsonx.datawatsonx.ai

☰ 코퍼스

전자금융거래법의 접근매체와 감독기관의 관계는?

전자금융거래법은 접근매체를 통해 전자금융거래를 수행하며, 이를 감독하는 기관은 금융위원회이며, 검사는 금융감독원이 수행한다. 또한, 전자지급거래의 청산·결제에 대해서는 한국은행이 감시(oversight) 기능을 수행한다.

관련 문서: 전자금융거래법#0

전자금융거래법#0전자금융감독규정#0특정금융정보법 (자금세탁방지) #0마이데이터 (본인신용정보관리업) #0마이데이터 (본인신용정보관리업) #1특정금융정보법 (자금세탁방지) #1전자금융거래법#1전자금융감독규정#1

TOOLS● vector search● knowledge graph● sql / iceberg

● Retrieved passages 8

● Knowledge subgraph 32

마이데이터 service접근매체 medium전자금융감독규정 regulation전자금융거래법 law전자금융거래 transaction금융회사 entity금융감독원 organization개인정보 보호법 law전자금융업자 entity이상거래담지시스템 FDS (technology)5·5·7 guideline접근통제·계정관리 technology정보보호최고책임자 CISO (role)고객원금거래보고 CTR (procedure)금융보안원 regulatory_body금융위원회 organization

마이데이터 — related_to → 접근매체

전자금융감독규정 — based_on → 전자금융거래법

데모 1c — sql 경로

 **Lakehouse Assistant**
Agentic RAG · Knowledge Graph · watsonx

OpenSearchAstraDBwatsonx.datawatsonx.ai

☰ 코퍼스

인덱싱된 문서는 총 몇 개이고 각각 체크 수는?

인덱싱된 문서는 총 8개이고, 각각의 체크 수는 다음과 같습니다:

- Docling README: 9개
- Langflow README: 6개
- 마이데이터(본인신용정보관리업): 2개
- 신용정보법: 2개
- 전자금융거래법: 2개
- 개인정보 보호법: 2개
- 특정금융정보법(자금세탁방지): 2개
- 전자금융감독규정: 2개

TOOLS

- * vector search
- * knowledge graph
- * **sql / iceberg**

• Corpus / SQL rows 8

Docling README	https://raw.githubusercontent.com/docling-project/docling/main/README.md	9	14	14
Langflow README	https://raw.githubusercontent.com/langflow-ai/langflow/main/README.md	6	15	15
마이데이터(본인신용정보관리업)	/corpus/03_mydata.md	2	11	9
신용정보법	/corpus/02_credit.md	2	11	12
전자금융거래법	/corpus/04_efta.md	2	11	10
개인정보 보호법	/corpus/01_pipa.md	2	12	10
특정금융정보법(자금세탁방				

Ask the lakehouse...

➤

데모 2 ★ — No-code 즉석 적재 (하이라이트)

RAG의 본질을 UI 클릭만으로 시연

1. 적재 전 질문 — "클라우드 보안인증(CSAP)는 어느 기관이 평가하나?"
→ ❌ 환각 (코퍼스에 없어 엉뚱한 답/가짜 인용)
2. ⚙️ 코퍼스 드로어 → 텍스트 탭 → 제목+본문 붙여넣기 → 추가 적재
→ "적재 완료" 토스트, 목록 8 → 9
3. 같은 질문 재질의 → ✅ 정확 + 새 문서 인용
4. 🗑️ 삭제 → 같은 질문 → 다시 환각으로 원복

입력 3종: 텍스트(주력) · URL(docling fetch) · 파일(PDF/DOCX)

발표 노트: "이게 핵심입니다. 코퍼스에 없는 걸 물으면 모델이 지어냅니다. 그 자리에서 문서를 붙여넣고 다시 물으면 정확히 답하고 인용까지 합니다. 지우면 다시 환각으로 돌아갑니다. '근거가 있으면 정확, 없으면 환각'을 30초 만에 보여주는 거죠."

데모 2 — 코퍼스 관리 드로어

The screenshot displays the Lakehouse Assistant interface with a '코퍼스 관리' (Corpus Management) drawer open on the right. The drawer has three tabs: '텍스트' (Text), 'URL', and '파일' (File). The '텍스트' tab is active, showing a form to add new documents with fields for '제목' (Title) and '문서 본문' (Document Content). Below the form is a '+ 추가 적재' (Add Load) button. The drawer also lists existing documents with their file paths and chunk/edge counts.

Document Name	File Path	Chunks	Ents	Edges
개인정보 보호법	/corpus/01_pipa.md	2	12	10
신용정보법	/corpus/02_credit.md	2	11	12
마이데이터(본인신용정보관리업)	/corpus/03_mydata.md	2	11	9
전자금융거래법	/corpus/04_efta.md	2	11	10
전자금융감독규정	/corpus/05_fsec.md	2	14	15
특정금융정보법(자금세탁방지)	/corpus/06_am1.md	2	16	15

설명가능성 — 근거를 어떻게 보여주나

- **tool-trace** 배지 — 이 답에 vector / graph / sql 중 무엇을 썼는지
- 인용 칩 — 답변 속 주장 ↔ 출처 문서 연결
- 근거 패널 (펼치기)
 - Retrieved passages — 청크별 유사도 바
 - Knowledge subgraph — 엔티티 + 엣지 트리플
 - Corpus / SQL rows — Iceberg 테이블
- 출처 링크 — URL 문서는 원문, 직접작성 md는 `/corpus/*.md` 서빙으로 열림

발표 노트: "신뢰의 핵심은 '왜 이렇게 답했는지'를 보여주는 겁니다. 배지로 도구를 보여주고, 패널로 실제 근거를 보여주고, 출처는 클릭하면 원문이 열립니다. 블랙박스가 아닙니다."

심화 ① — KG를 NoSQL(AstraDB)에 담은 스키마

왜 그래프DB가 아니라 NoSQL? 코퍼스가 작고(수십 문서·엣지), 질의는 "엔티티 주변 1홉 서브그래프"뿐 → 멀티홉 순회 엔진 불필요. 엣지를 통째로 읽어 앱에서 필터·랭킹하면 충분.

단일 컬렉션 **kg** — **kind** 필드로 노드/엣지 구분 (AstraDB Data API JSON 문서)

```
{ "_id": "<doc_id>:e:3", "kind": "entity", "doc_id": "...",  
  "name": "마이데이터", "type": "service", "norm": "마이데이터" }  
  
{ "_id": "<doc_id>:r:5", "kind": "edge", "doc_id": "...",  
  "src": "마이데이터", "rel": "based_on", "dst": "신용정보법",  
  "src_norm": "마이데이터", "dst_norm": "신용정보법" } // 정규화 트리플
```

- **_id** = **doc_id** 스코프 → 문서별 delete→insert로 먹등 재적재 · **type** = 엔티티 분류
- ***_norm** = 정규화 키(별칭 병합용) · **rel** = 통제 어휘 9종(snake_case)만 허용(**_clean_rel** 이 한글 조사를 **related_to** 로 강등)

발표 노트: "자주 받는 질문이죠 — 왜 그래프DB가 아니냐. 답은 '필요 없어서'입니다. 엣지를 평범한 JSON 문서로 평면 저장합니다. 한 컬렉션에 kind로 노드와 엣지를 같이 두고, 이름은 정규화 키를 미리 박아둡니다."

심화 ② — KG 적재·질의 동작

적재 (write) — 문서 1건 → 같은 doc_id로 삭제 후 재삽입

```
granite가 {entities, edges} JSON 추출 → norm/src_norm/dst_norm 부착  
→ astra_delete_all(doc_id) → insertMany # 멍등
```

질의 **tool_graph** (read) — 그래프 순회 대신 전체 로드 → 앱에서 필터·랭킹

```
astra_find_all(kg) 전체 KG 로드 (Data API 20/page 페이지네이션)  
→ 엣지 점수 = 질문에 엔티티 언급(+2) + 벡터시드 문서(+1)  
→ score>0 보존 (없으면 최고차수 서브그래프 폴백)  
→ *_norm 으로 노드 병합 · 중복엣지/자기루프 제거 → 칩 + 트리플
```

- 벡터시드: OpenSearch로 관련 문서를 먼저 찾아 그 문서 엣지에 가점 → **KG+벡터 결합**, 엔티티 해소(별칭 병합)는 그래프엔진 대신 정규화 키로
- 트레이드오프: 멀티홉 경로탐색 없음(1홉 한정) → 대규모 KG엔 부적합, 그때 진짜 그래프DB

발표 노트: "질의는 순회가 아니라 '읽고 거르기'입니다. KG를 통째로 읽어 질문에 나온 엔티티 엣지에 +2, 벡터로 걸린 문서 엣지에 +1을 줘 상위만 남깁니다. 작은 KG라 가능한 설계고, 커지면 그래프DB로 갑니다."

품질 개선 — 적용 완료

- 하이브리드 검색 — 벡터(의미) + BM25(정확매칭) **RRF** 융합
→ 약어·법령명(STR · VASP · CISO)에 강함
- **cjk** 분석기 — 한글을 bigram 토큰화 → 한글 BM25 recall 대폭 개선
(nori는 operator 관리 클러스터라 설치 불가 → 내장 cjk로 대체)
- **KG** 정규화 — 엔티티명 정규화 · 별칭 병합(마이데이터 = 마이데이터 사업자)
관계 라벨 snake_case 강제(조사 "의/는" 제거) · 자기루프 제거

발표 노트: "PoC를 실제로 쓸 만하게 만든 디테일들입니다. 한글 검색이 특히 까다로웠는데, 형태소 분석기를 못 깔아서 내장 cjk로 우회했고, 지식그래프는 같은 개념이 다른 노드로 쪼개지는 걸 정규화로 합쳤습니다."

배포 — 이미지 빌드 없이

- 내부 레지스트리 **Removed** 환경 → 이미지 빌드 불가
- 해법: **ConfigMap + 런타임 pip-install** 패턴
 - 코드/정적파일을 ConfigMap으로 마운트 → `ubi9/python-312` 가 기동 시 설치·실행
- 단일 컨테이너 FastAPI — 모든 자격증명은 백엔드(`rag-secrets`)에 보관
- 그래프 질의가 30s 초과 → 라우트 타임아웃 `120s` 설정

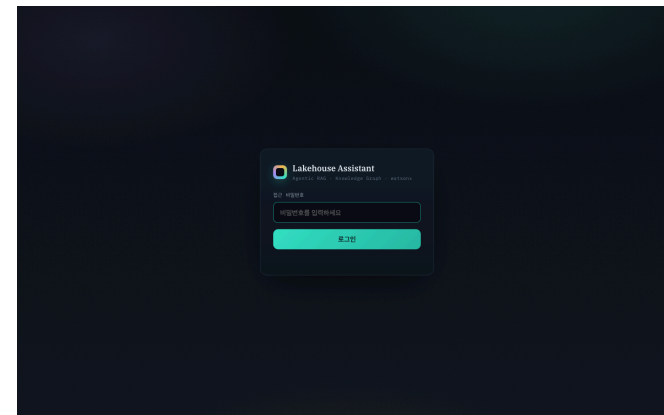
```
oc -n genai-apps create configmap rag-code --from-file=*.py --dry-run=client -o yaml | oc apply -f -
oc -n genai-apps rollout restart deploy/rag-ui # 이미지 빌드 없음
```

발표 노트: "제약이 많은 환경이었습니다. 내부 레지스트리가 막혀 있어서 이미지를 못 만들었어요. 그래서 코드를 ConfigMap으로 올리고 컨테이너가 뜰 때 pip로 설치하는 패턴을 썼습니다. 제약 환경에서의 현실적인 배포 패턴입니다."

보안 — 공유 비밀번호 게이트

- 채팅 UI는 공유 비밀번호 로그인으로 보호 (서명된 세션 쿠키, HMAC)
- `rag-secrets` 의 `APP_PASSWORD` 키로 설정 — 키 없으면 인증 없이 열림 (하위호환)
- 미인증: 페이지 → `/login` 리다이렉트, API → `401`
- ⚠ 적재/삭제는 로그인 후 누구나 가능(데모 목적)
→ 운영엔 사용자별 인증 + 트랜잭션 보강 필요

발표 노트: "공개 데모라 공유 비밀번호 로그인만 붙였습니다. 적재·삭제는 로그인 후 누구나 가능 — 운영으로 가려면 사용자별 인증·트랜잭션이 더 필요합니다."



알아둘 한계 (데모 수용 범위)

- 무인증 쓰기 — 적재/삭제 공개. 운영 시 인증·outbox 트랜잭션 필요
- 라우터 비결정성 — 관계형 질문을 가끔 vector로 분류
→ "관계/지식그래프 엣지로"라고 명시하면 graph 확실히 커짐
- 그래프 순회 아님 — 1홉 서브그래프 수준(멀티홉 경로탐색 X). AstraDB는 비벡터 NoSQL
- **docling-graph** 미사용 — KG는 granite LLM 추출. docling-graph는 스캔PDF/복잡표용 옵션

발표 노트: "정직하게 한계도 말씀드립니다. 라우터가 가끔 틀리고, 그래프는 한 홉짜리 단순 서브그래프입니다. 멀티홉 경로 탐색이 필요하다면 진짜 그래프 DB가 필요합니다. 데모 범위에서 수용한 트레이드오프들입니다."

핵심 메시지

- **Agentic RAG** = 검색 하나가 아니라, 질문에 맞는 도구를 고르는 것
- **Explainability** = 답뿐 아니라 근거를 보여줘야 신뢰가 생긴다
- **AI-Ready Context** = 같은 문서를 벡터·그래프·SQL 세 형태로 준비
- **watsonx + OpenShift** = 규제 산업에서도 거버넌스 가능한 스택

데모: `https://rag-ui-genai-apps.apps.<CLUSTER_DOMAIN>`

코드: `github.com/kiyeonjeon21/techxchange-ai-ready-context`

발표 노트: "정리하겠습니다. 에이전틱 RAG는 도구 선택이고, 설명가능성은 신뢰의 조건이고, 그러려면 데이터를 세 가지 형태로 미리 준비해 뒀야 합니다. 그리고 이 모든 걸 watsonx와 OpenShift 위에서 거버넌스 가능한 형태로 했습니다. 감사합니다."

Q&A / 부록 — 기술 노트

- watsonx.data Presto는 **PrestoDB** (`presto-python-client` , trino 아님)
- OpenSearch: faiss 없음 → **lucene kNN**, 한글 BM25는 **cjk 분석기**
- KG 추출: granite LLM(`extract_kg`) — 관계 `snake_case` 정규화 + 별칭 병합
- 모델: granite-3-8b-instruct (채팅/라우팅/합성) + granite-embedding-278m-multilingual (768d)
- 파싱: docling-serve `/v1/convert/source` (URL) · `/v1/convert/file` (multipart)

상세: [docs/architecture.pdf](#) · [docs/wxdata-install-journey.md](#) · [NOTE.md](#)

발표 노트: "질문 받겠습니다. 기술적으로 더 깊은 내용은 이 부록과 저장소의 NOTE.md, 구축 여정 문서에 정리해 뒀습니다."